

Project

1. Context

In modern parliamentary democracies, the process of forming a government is rarely a simple matter of who has the most votes. Instead, political parties attempt to form coalitions. Each party enters negotiations with a set of “core values” and “constraints”. For example, a “Green Party” might have an axiom that carbon must be taxed (*i.e.*, `CarbonTax = True`), while a “Labour Party” might have a constraint that the minimum wage must be increased (*i.e.*, `MinWageIncrease = True`). If a third party, the “Conservatives,” holds a constraint that carbon taxes must not be increased (*i.e.*, `CarbonTax = False`), a logical contradiction emerges. In the real world, this leads to *hung parliaments* or failed states. In this project, students will treat the legislature as a computational system where stability is a provable property.

2. Project Statement

Objective: The objective of this project is to develop a software in Python that takes as input the policy demands and seat counts of various political parties, and use a custom-built SAT solver to determine if a stable governing majority can be formed or not.

You will represent a political system where:

- **The Universe (U):** A set of all possible legislative policies.
- **The Parties (P):** A set of organizations, each defined by a function that maps policies to a preference: $f : Policy \rightarrow \{1, -1, 0\}$ (*i.e.*, Support, Oppose, or Abstain) and the amount of seats they represent.
- **The Constitution:** A set of First-Order Logic rules that define a valid government (*e.g.*, “The sum of seats of parties in the coalition must be greater than 50% of the total”).

```

1 # Initialize the System
2 # where green, labour, industrial are the political preferences' (policies)
3 system = PoliticalSAT(seats, [green, labour, industrial])
4
5 # Run the Solver; looks for >=50 percent of the seats in the coalition
6 decision = system.solve_stability(50/100)
7
8 print(decision) # a list of decisions regarding policies

```

In particular, your solver should receive a vector `seats` where $seats(i) \in \mathbb{N}_{>0}$ represents the amount of seats of the i th party; a matrix `policies` where $policies(i, j) \in \{-1, 0, 1\}$ represents the stance of the i th party regarding the j th policy; and, a value $p \in [0, 1]$ that represents the fraction of the total seats in the coalition for it to be considered a valid government.

3. Technical Deliverables

- **The Policy Encoder:** A module that takes the "Rules of the Land" (FOL) and "Party Demands" and translates them into Disjunctive Normal Form (DNF).
- **The Custom SAT Solver:** A Python-based implementation of the solver.
- **The Stability Report:** If a solution exists, your program must output the specific list of policies. If no solution exists, the program must prove that the constraints are Unsatisfiable.

4. Implementation Constraints

- **Use of artificial intelligence is forbidden:** Any use of AI in the completion of this project will result in its cancellation and possible disciplinary action.
- **Good programming practices:** The use of `continue`, `break`, or `pass` is not permitted. If a function returns a value, there should only be one return, which must be at the end of the definition.
- **The use of native functions such as `eval` or external libraries is forbidden:** The implementation must be carried out using only native Python functionalities. The objective is for students to understand all the functions and definitions involved.
- **Report:** In addition to the code, a document must be submitted that briefly explains the ideas that led to the solution of the problem.

5. Evaluation

This project will be evaluated based on

- The implementation is correct. To test this, there exists a straightforward method to determine if a given assignation is correct. If there exists no assignation, the code will be tested against a model solution (they should both reach the same conclusion on its feasibility).
- The report conveys properly the ideas used on the code.
- The program allows executing certain test cases in a comfortable time (< 5 seconds per case). The more the “difficulty” of the cases it solves, the greater the compliance on this item. The specifics, however, are still under discussion.

Notes: The policy stances will be generated randomly and uniformly over all $3^{n \cdot p}$ distinct such matrices (where p is the amount of policies and n the amount of parties). The amount of seats will be generated uniformly over all 20^n vectors having $1 \leq \text{seats}(i) \leq 20$.

6. Repo

Feel free to use the code in the following repository to test your code. It is not a requisite of this project to use it, but it can(?) help you. The only requisite is making sure the names of the solver class and its methods align to help with the verification of the program. https://github.com/osvarp/discreet_math_20261_project